Project Proposal Template

Agentic AI Applications Course

Team Number:

Team Member Name: MOHAMMED SAHIL ABDUL

Proposed Project Title: Agentic AI-Based Multimodal Nutrient Deficiency Detection System Using Medical Image Analysis

1. Problem Statement & Use Case

What real-world problem are you solving?

Micronutrient deficiencies such as Iron deficiency, Vitamin B12 deficiency, Folic Acid deficiency, Protein deficiency, and Zinc deficiency affect millions of people worldwide. Early detection usually requires blood tests, which can be expensive, time-consuming, and inaccessible in rural or low-resource healthcare environments.

However, many nutrient deficiencies produce visible physical symptoms in areas such as the eyes, tongue, and nails, including pallor, discoloration, glossitis, and nail abnormalities.

Our system aims to automatically detect possible nutrient deficiencies using computer vision analysis of medical images.

Who is the target user / beneficiary?

Primary Users:

- Healthcare practitioners in remote clinics
- Telemedicine platforms
- Nutritionists and dieticians

Secondary Users:

- Individuals performing preliminary health checks
- Medical researchers studying micronutrient deficiencies
- NGOs working in rural health screening

What currently happens (manual process, existing tools, pain points)?

Currently:

1. Nutrient deficiency diagnosis requires blood tests and laboratory analysis.
2. Patients must visit hospitals or diagnostic centers.
3. Medical professionals visually inspect symptoms manually but lack automated diagnostic assistance.

Pain Points:

- Cost of laboratory testing
- Limited access to healthcare infrastructure in rural areas
- Delay in diagnosis
- Lack of automated tools for early screening

Why is this a good fit for an Agentic AI solution?

A simple ML model can detect patterns from images, but an Agentic AI system can orchestrate multiple reasoning and analysis steps automatically, such as:

- Collecting multimodal inputs (eyes, tongue, nails)
- Performing preprocessing and feature extraction

- Running multiple ML models
- Validating prediction confidence
- Cross-checking symptoms with medical knowledge
- Generating explanations and recommendations

An agentic system can plan, analyze, verify, and refine results autonomously, making it much more powerful than a single model prediction or simple LLM response.

2. Justify that your application is Agentic

Our application requires several agentic capabilities.

Core Agentic Capabilities

1. Tool Use

Agents will use multiple tools:

- Computer Vision processing (OpenCV)
- Feature extraction modules
- ML model inference
- Medical knowledge retrieval
- Confidence evaluation tools

2. Planning & Task Decomposition

The system must break down the diagnosis process into steps:

1. Validate uploaded images
2. Preprocess images

3. Extract features
4. Run ML models
5. Cross-validate predictions
6. Generate medical explanation

3. Reflection / Self-Correction

The agent evaluates prediction confidence.
If results are inconsistent (e.g., eye predicts iron deficiency but tongue does not), the agent:

- Re-analyzes the images
- Requests better input
- Adjusts predictions

4. Multi-Agent Collaboration

Different agents specialize in tasks:

- Image analysis
- ML prediction
- Medical reasoning
- Final diagnosis report generation

What would fail if this were a single LLM call?

A single LLM prompt would:

- Not process real medical images
- Not run machine learning models
- Not verify prediction reliability
- Not coordinate multiple diagnostic signals

Therefore, an agentic system is required to coordinate tools, models, and reasoning loops.

Expected Agentic Behavior Example

Example workflow:

1. User uploads eye, tongue, and nail images.
2. Image Analysis Agent processes images.
3. ML Prediction Agent runs Random Forest and SVM models.
4. Medical Reasoning Agent cross-checks predicted symptoms.
5. Reflection Agent checks confidence levels.
6. If confidence is low → system requests clearer images.

3. High-Level Solution Overview

The system will use a multi-agent architecture where each agent performs a specialized role.

Agents and Roles

1. Input Validation Agent

- Verifies image quality
- Ensures correct image types (eye, tongue, nail)

2. Image Analysis Agent

- Performs preprocessing
- Feature extraction using:
    - HOG
    - LBP
    - Color Histograms

3. Prediction Agent

Runs trained ML models:

- Random Forest
- Support Vector Machine

Outputs predicted nutrient deficiencies.

4. Medical Reasoning Agent

- Interprets model predictions
- Cross-references medical symptom patterns

5. Reflection Agent

- Checks confidence scores
- Re-analyzes if inconsistencies occur

6. Report Generation Agent

Generates final diagnostic report with:

- Predicted deficiency
- Confidence score
- Suggested medical follow-up

Agentic Patterns Used

- Plan-and-Execute
- Multi-Agent Collaboration
- Reflection / Self-Critique Loop
- Tool Usage

Memory Strategy

- Short-term memory: conversation context
- Vector database: storing medical symptom patterns
- Prediction logs: historical results for evaluation

High-Level Flow

User Input (Images)

|

▼

Input Validation Agent

|

▼

Image Analysis Agent

|

▼

Feature Extraction

|

▼

ML Prediction Agent

|

▼

Medical Reasoning Agent

|

▼

Reflection Agent

|

▼

Report Generation Agent

|

▼

Final Diagnosis Output

4. Proposed Technology Stack

LLM Provider

- OpenAI GPT-4 / GPT-4o
- Used for reasoning, explanations, and agent coordination.

Reason:
High reasoning ability and tool integration capability.

Core Framework

LangGraph

Reason:

- Supports multi-agent workflows
- Allows tool-based reasoning
- Enables reflection loops

Alternative: CrewAI

Tools

Custom Tools:

- Image preprocessing tool
- Feature extraction module
- ML model inference tool
- Medical knowledge retrieval tool

External Tools:

- Python code execution
- Scikit-learn models

Vector Database / Memory

ChromaDB

Used for:

- storing symptom knowledge
- medical reference data
- model results history

Machine Learning Models

- Random Forest
- Support Vector Machine

Computer Vision

- OpenCV
- HOG
- LBP
- Color Histograms

Frontend / User Interface

Streamlit or Gradio

Features:

- Image upload
- Webcam capture
- Real-time diagnosis
- Report visualization

Evaluation Tools

- Custom Python evaluation scripts
- Accuracy metrics
- Manual evaluation rubric

Deployment

- Hugging Face Spaces
  or
- Render

5. Success Metrics & Evaluation Plan

To evaluate system performance, we will track both quantitative and qualitative metrics.

Quantitative Metrics

1. Prediction Accuracy
   Percentage of correct nutrient deficiency predictions.
2. Precision
   Accuracy of positive predictions.
3. Recall
   Ability to detect true deficiencies.
4. F1 Score
   Balanced metric between precision and recall.
5. End-to-End Response Time
   Time taken for diagnosis.
6. Model Confidence Score

Qualitative Evaluation

We will evaluate:

- Accuracy of medical explanations
- Clarity of generated diagnostic reports
- Usability of the interface

A small user study with 5–10 users (students or healthcare volunteers) will be conducted.

Test Cases

We plan to evaluate the system using 20–30 test cases consisting of:

- Eye images
- Tongue images
- Nail images

Anticipated Failure Modes

Possible issues:

- Low-quality images
- Mixed symptoms across modalities
- Dataset bias

Evaluation will measure how often these errors occur.

6. Risks, Challenges & Mitigation Plan

Risk 1: Poor Image Quality

Users may upload blurred images.

Mitigation:
Image validation agent will request better images.

Risk 2: Limited Dataset

Medical image datasets may be small.

Mitigation:

- Data augmentation
- Transfer learning
- Synthetic dataset expansion

Risk 3: Model Misclassification

Symptoms may overlap across deficiencies.

Mitigation:

- Ensemble ML models
- Cross-modal verification

Risk 4: Agent Loop Errors

Agents might repeatedly reprocess inputs.

Mitigation:

- Maximum iteration limits
- Timeout controls

Risk 5: Medical Reliability Concerns

System should not replace professional diagnosis.

Mitigation:

- Add disclaimer
- Suggest medical consultation

7. Timeline & Milestones

By March 25 (Demo 1)

- Develop ML models
- Implement feature extraction pipeline
- Build single-agent system with tool usage
- Basic Streamlit UI

By April 10 (Document 2)

- Design full multi-agent architecture
- Implement LangGraph workflow
- Initial evaluation results
- Documentation of system design

By April 22 (Demo 2)

- Multi-agent MVP working
- Reflection agent implemented
- Evaluation dashboard
- Deploy prototype

By May 5 (Final Submission)

- Fully functional system
- Improved UI
- Final evaluation results
- Project presentation and report

ine.